

PROJECT DESIGN DOCUMENT

I. Introduction (Giới thiệu chung)

1.1. Mục tiêu hệ thống (System Objectives)

Mục tiêu của thiết kế là xây dựng một hệ thống mô phỏng sổ cái phân tán (Distributed Ledger) có khả năng duy trì tính nhất quán dữ liệu giữa các thực thể độc lập, ngay cả khi hệ thống chịu ảnh hưởng bởi hai loại lỗi nghiêm trọng cùng lúc:

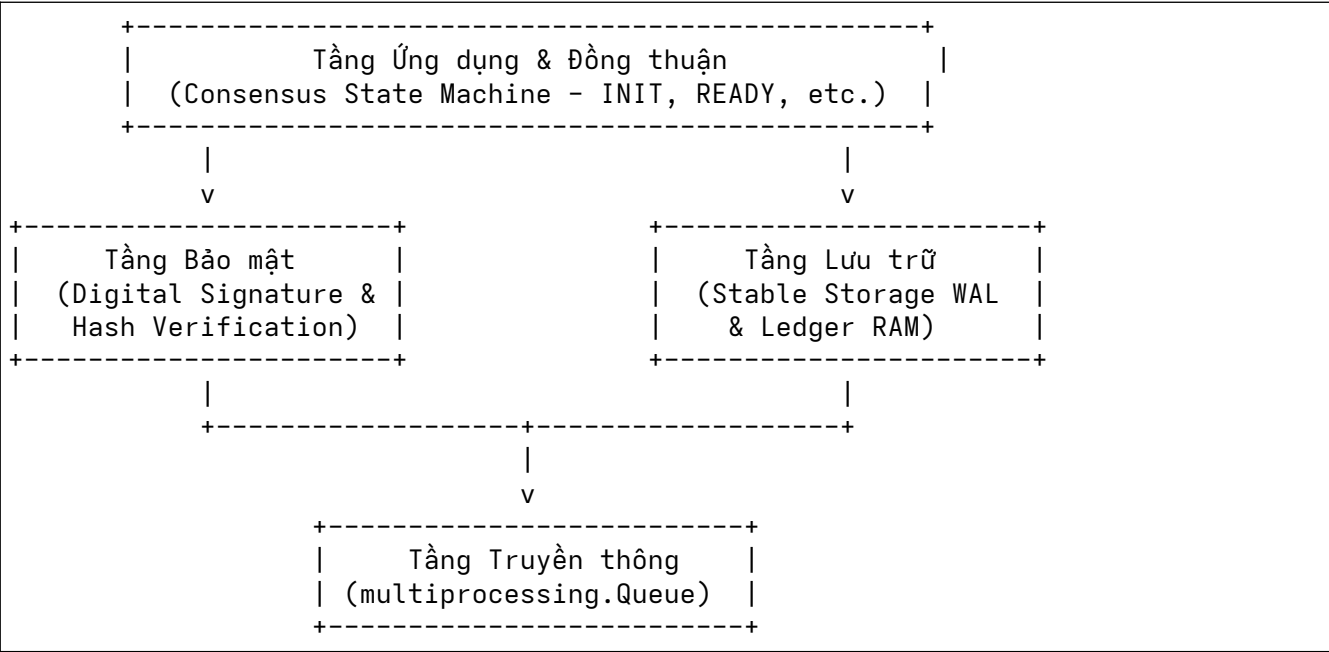
1. Lỗi Byzantine (Byzantine Faults): Leader (Node 0) cố ý hành xử độc hại, gửi thông điệp đề xuất chứa digest mâu thuẫn (Equivocation) đến các node khác nhau hoặc trì hoãn gửi tin nhắn nhằm phá vỡ đồng thuận.
2. Lỗi Crash & Phục hồi (Fail-Stop and Recovery): Một node trung thực (Node 1) bị sập nguồn đột ngột (container stop) trong quá trình đồng thuận, sau đó tự khởi động lại và khôi phục trạng thái nhất quán bằng RocksDB WAL và checkpoint.

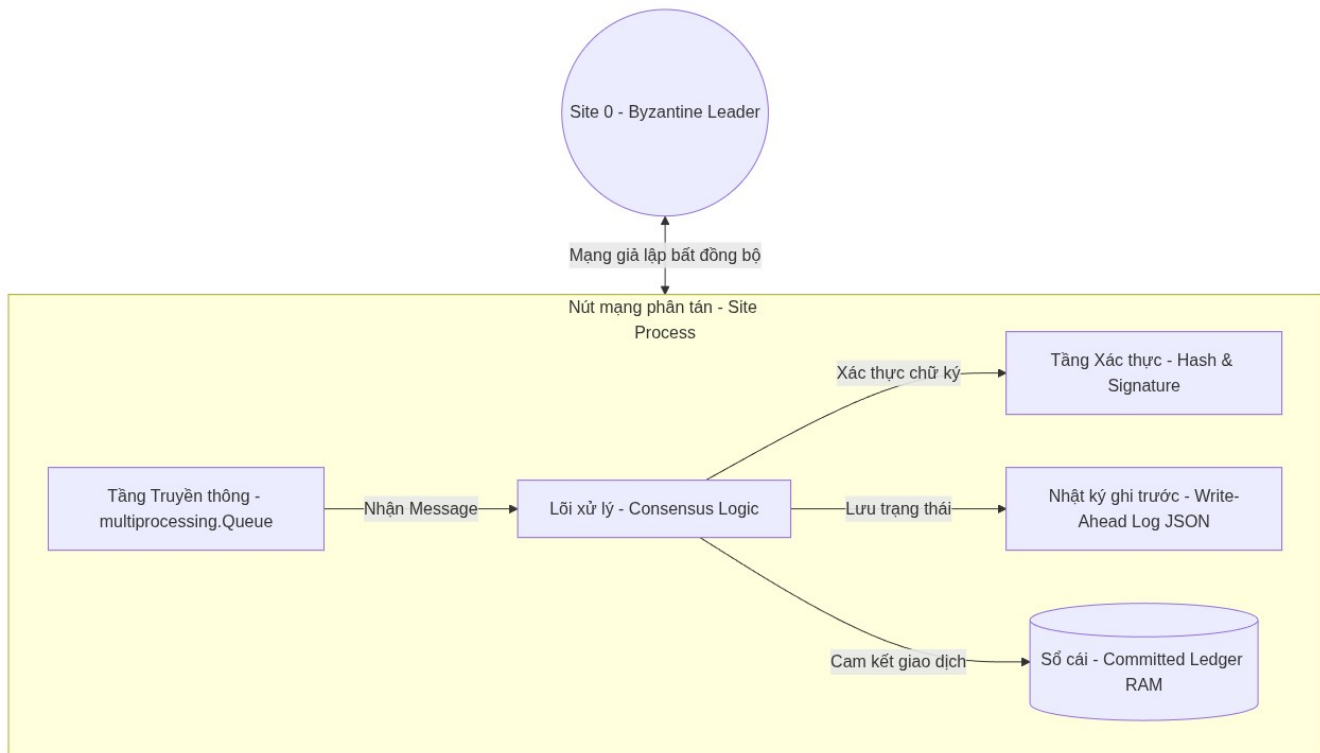
1.2. Các giả định hệ thống (System Assumptions)

- Số lượng nút mạng: Hệ thống bao gồm $N = 4$ tiến trình (Site 0 đến Site 3).
- Giới hạn lỗi: Số lượng nút Byzantine tối đa là $f = 1$ Số lượng nút trung thực tối thiểu là 3.
- Mô hình mạng: Mạng truyền thông bất đồng bộ (Asynchronous network). Độ trễ của thông điệp là ngẫu nhiên nhưng đảm bảo thông điệp sẽ được phân phối đến đích (không bị mất gói tin vĩnh viễn).
- Mô hình lỗi crash: Các nút trung thực có thể bị crash bất kỳ lúc nào nhưng khi khôi phục, chúng sẽ hoạt động chính xác theo giao thức nhờ dữ liệu lịch sử lưu trên bộ nhớ bền vững (Stable Storage).

II. System Architecture (Kiến trúc hệ thống)

Kiến trúc phân tầng của một nút mạng (Site) được thiết kế để tách biệt rõ ràng giữa truyền thông, logic nghiệp vụ, bảo mật và lưu trữ.





2.1. Các thành phần chính trong kiến trúc:

1. Tiến trình độc lập (Site Process): Mỗi Site là một tiến trình hệ điều hành riêng biệt, sở hữu bộ nhớ RAM độc lập, không chia sẻ biến toàn cục để giả lập chân thực môi trường phân tán.
2. Giao tiếp TCP Sockets & Connection Pool: Các node kết nối trực tiếp qua các socket TCP dài hạn. Connection Pool duy trì kết nối liên tục, tự động kết nối lại khi lỗi, và truyền các gói tin JSON được phân tách bằng dòng '\n'.
3. Lưu trữ cục bộ RocksDB (LSM-Tree): Mỗi node sử dụng RocksDB lưu trữ WAL, sổ cái Ledger và World State. Định dạng LSM-Tree giúp đọc/ghi đĩa cực nhanh, đảm bảo tính bền vững (durability) của hệ thống phân tán.
4. Sổ cái Ledger (Committed Ledger): Sổ cái được lưu trữ bền vững trong RocksDB dưới prefix ledger::<tx_id>, ghi lại toàn bộ chuỗi các giao dịch đã được đồng thuận cam kết (COMMITTED).
5. Tầng mật mã học Ed25519 (Cryptography Layer): Chịu trách nhiệm tạo cặp khóa public/private key, ký số các thông điệp truyền mạng và xác thực chữ ký số Ed25519 thực tế để chống giả mạo danh tính (spoofing) và đảm bảo tính toàn vẹn (integrity).

III. Data Structures & Storage (Cấu trúc dữ liệu và Lưu trữ)

3.1. Cấu trúc thông điệp giao dịch (Transaction Schema)

Mỗi giao dịch đề xuất có cấu trúc dữ liệu như sau:

- tx_id: Định dạng băm chuỗi (hex string digest) hoặc số nguyên tăng dần của giao dịch.
- data: Nội dung chi tiết của giao dịch tài chính (người gửi, người nhận, số tiền, timestamp) kèm signature (chữ ký số Ed25519 của client).

3.2. Cấu trúc các loại thông điệp mạng (Network Message Schema)

Hệ thống trao đổi các loại thông điệp PBFT chính qua mạng TCP Sockets:

1. MSG_CLIENT_REQUEST: Client gửi giao dịch (chứa chữ ký Ed25519 của client).

2. MSG_PRE_PREPARE: Leader đề xuất giao dịch, gán view, seq, và digest của giao dịch.

```
{
  "type": "VOTE",
  "sender": 1,
  "tx_id": 1,
  "vote": "COMMIT",
  "signature": "SIG_SITE_1"
}
```

3. MSG_PREPARE: Các node phát chéo phiếu chuẩn bị để đạt quorum $2f+1$.

4. MSG_COMMIT: Các node phát chéo phiếu cam kết thực thi sau khi nhận đủ $2f+1$ phiếu PREPARE.

```
{
  "type": "RECOVERY_REQ",
  "sender": 1,
  "tx_id": 1,
  "signature": "SIG_SITE_1"
}
```

5. MSG_CHECKPOINT: Các node phát chéo state digest của sổ dư để thiết lập Stable Checkpoint.

6. MSG_VIEW_CHANGE & MSG_NEW_VIEW: Sử dụng để bầu Leader mới khi Leader cũ bị lỗi.

```
{
  "type": "RECOVERY_RESP",
  "sender": 2,
  "tx_id": 1,
  "vote": "COMMIT",
  "signature": "SIG_SITE_2"
}
```

3.3. Thiết kế Write-Ahead Log (WAL)

Thiết kế RocksDB Storage Schema:

Dữ liệu lưu trữ trong RocksDB sử dụng tiền tố khóa để phân mảnh:

- wal::::<seq>: Ghi nhật ký đồng thuận trạng thái giao dịch.
- state::- ledger::- checkpoint::

-

- 1.
- 2.
- 3.

Bạn nói rất chính xác. Trong đề cương phần 4 (Consensus Protocol Design), chúng ta cần thiết kế một mục riêng biệt để mô tả rõ ràng **Quy trình đồng thuận khi Leader trung thực** (Normal Consensus Flow), tách biệt hẳn với luồng lỗi phức tạp để người đọc (và giảng viên) dễ theo dõi cơ chế hoạt động cơ bản trước.

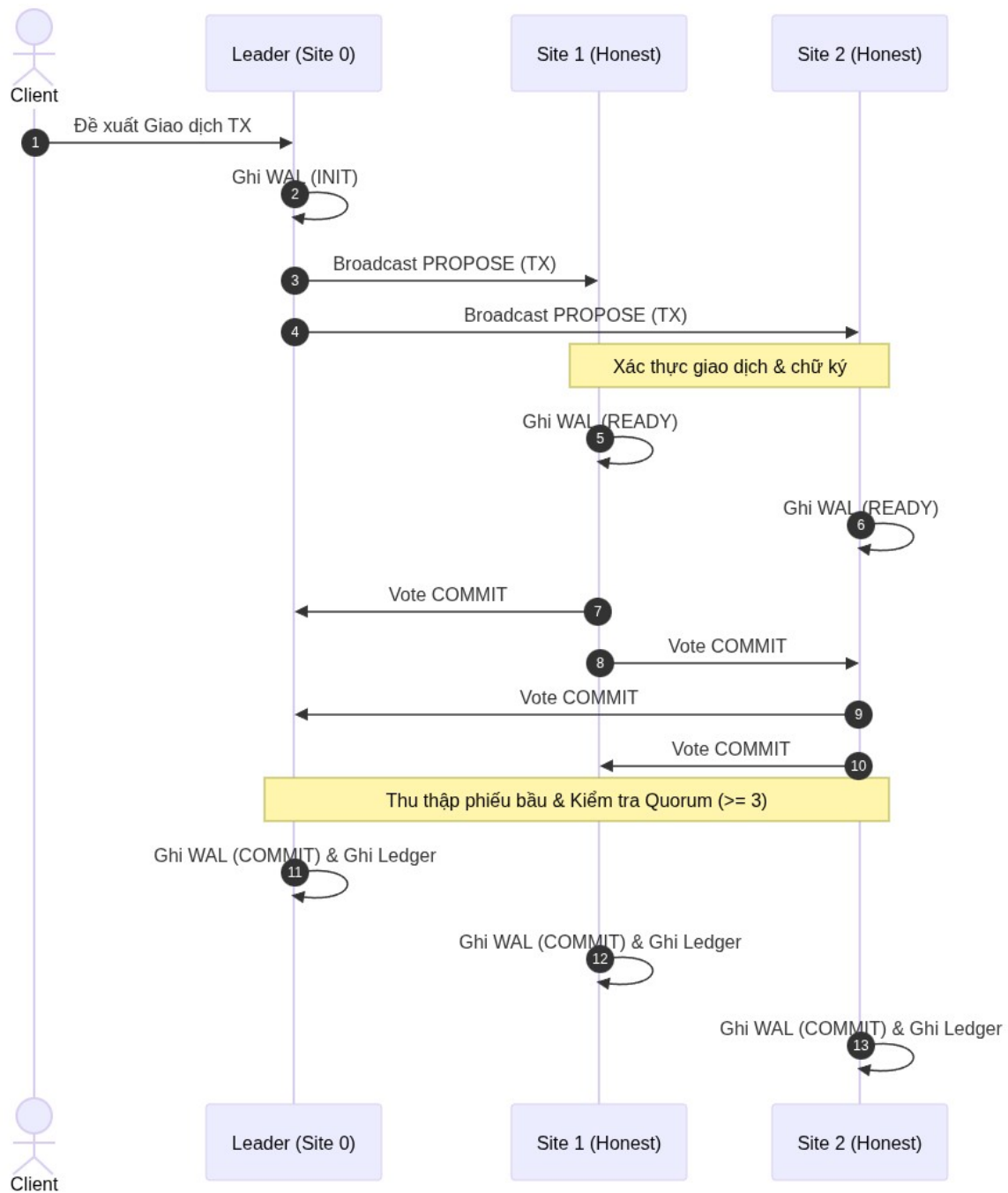
Dưới đây là thiết kế chi tiết cho mục này để bạn đưa vào tài liệu:

4.1. Quy trình đồng thuận khi Leader trung thực (Normal Consensus Flow)

Mục này mô tả luồng thông điệp và sự thay đổi trạng thái của các nút trong điều kiện lý tưởng: Điều phối viên (Site 0) hoạt động trung thực và không có sự cố mạng hay sập nguồn xảy ra.

4.1.1. Sơ đồ tuần tự thông điệp (Sequence Diagram)

Dưới đây là sơ đồ tuần tự mô tả các bước trao đổi thông điệp chéo giữa 4 Site (Site 0 là Leader trung thực, Site 1, 2, 3 là các Replica trung thực):



4.1.2. Các bước xử lý chi tiết (Step-by-Step Processing):

1. Bước 1: Propose (Đề xuất)

Leader (Node 0) nhận giao dịch từ client, xác thực chữ ký client, băm digest, gán số thứ tự seq và broadcast bản tin MSG_PRE_PREPARE.

- Leader (Site 0) nhận yêu cầu giao dịch mới từ client.
- Site 0 ghi trạng thái INIT vào nhật ký đĩa cứng [site_0.wal](#).

- Site 0 đính kèm chữ ký số giả lập `SIG_SITE_0` và broadcast thông điệp đề xuất đến Site 1, 2, 3.

2. Bước 2: Prepare & Broadcast (Chuẩn bị & Phát chèo)

Các node backup nhận PRE-PREPARE, xác thực chữ ký Leader, và broadcast bản tin `MSG_PREPARE` đến tất cả các node.

- Khi nhận được đề xuất, mỗi Replica (Site 1, 2, 3) gọi hàm `verify_signature()` để kiểm tra chữ ký số của Site 0.
- Các Replica kiểm tra tính hợp lệ của số dư giao dịch. Nếu hợp lệ, mỗi Replica ghi trạng thái `READY` kèm thông tin phiếu `vote: COMMIT` vào tệp WAL cục bộ của mình để đảm bảo tính bền vững (durability) trước khi gửi tin.
- Mỗi Replica tự ký thông điệp bằng chữ ký riêng và broadcast phiếu `COMMIT` của mình cho toàn bộ các nút khác trong mạng.

3. Bước 3: Decide & Commit (Quyết định & Cam kết)

- Bước 3: Commit & Execute (Cam kết & Thực thi)
Khi nhận đủ $2f$ phiếu `PREPARE` trùng khớp digest từ các node khác (tổng cộng $2f+1$ phiếu tính cả bản thân), node chuyển sang trạng thái `Prepared`, ghi WAL và broadcast `MSG_COMMIT`. Khi nhận đủ $2f+1$ phiếu `MSG_COMMIT` hợp lệ, node thực thi giao dịch, cập nhật số dư World State, ghi Ledger vào RocksDB.
- Vì tất cả các nút đều trung thực và gửi phiếu nhất quán, mỗi nút sẽ nhận được tối thiểu 3 phiếu bầu `COMMIT` hợp lệ (bao gồm phiếu từ Site 1, Site 2, Site 3).
- Do số lượng phiếu `COMMIT` đạt $3 \geq \text{Quorum}(3)$, mỗi nút thực hiện chuyển trạng thái:
 - Ghi trạng thái `COMMIT` vào tệp WAL trên đĩa.
 - Thêm giao dịch vào sổ cái `ledger` trên RAM.
 - Hoàn tất giao dịch đồng thuận thành công và sẵn sàng xử lý giao dịch tiếp theo.

4.1.3. Bảng trạng thái chuyển đổi của các nút (State Transition Table)

Bảng dưới đây tóm tắt trạng thái ghi nhận tại tệp WAL của các nút tại các thời điểm trong luồng đồng thuận bình thường:

Thời điểm	Trạng thái Site 0 (Leader)	Trạng thái Site 1	Trạng thái Site 2	Trạng thái Site 3
Bắt đầu	INIT	None	None	None
Sau khi đề xuất	READY	READY	READY	READY
Sau khi nhận đủ phiếu chèo	COMMIT	COMMIT	COMMIT	COMMIT
Kết quả Sổ cái	Cam kết	Cam kết	Cam kết	Cam kết

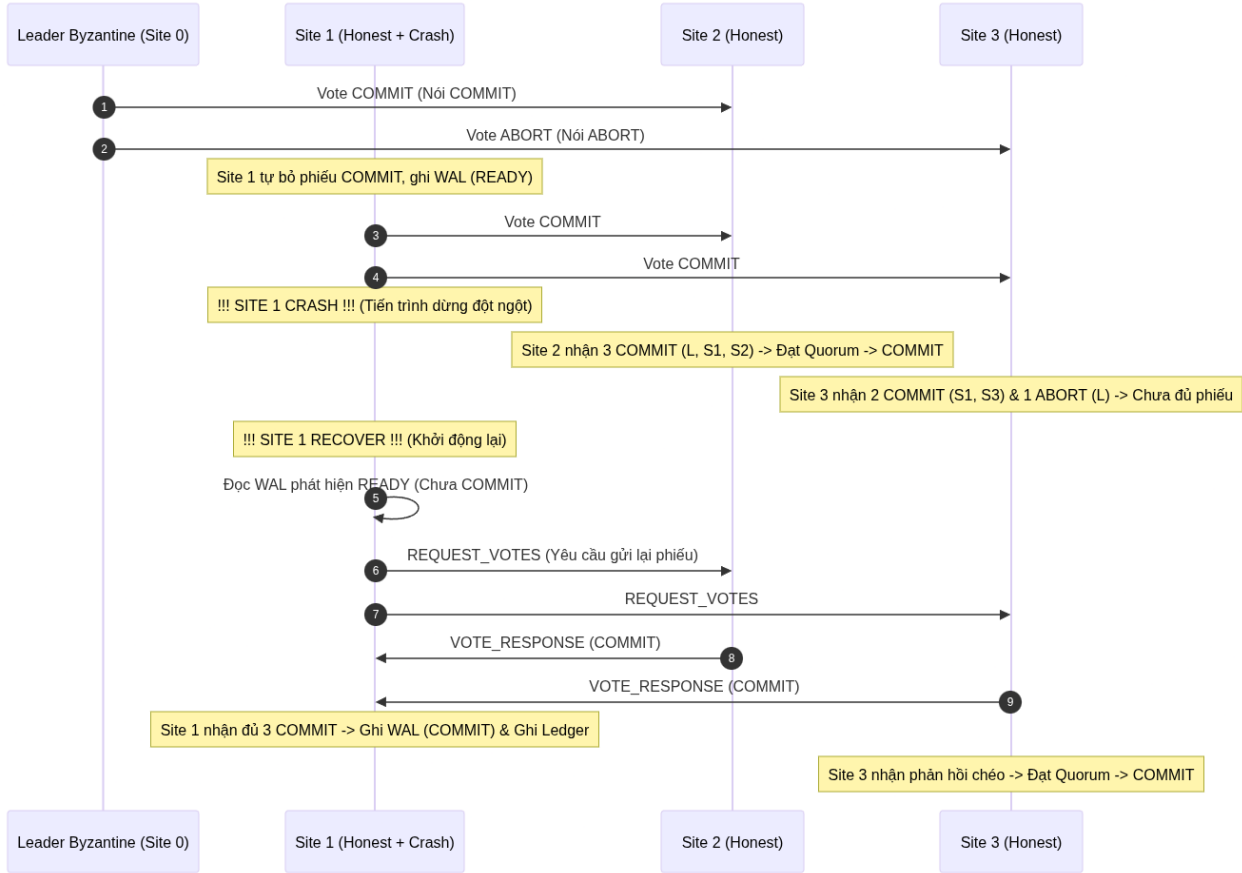
Thời điểm	Trạng thái Site 0 (Leader)	Trạng thái Site 1	Trạng thái Site 2	Trạng thái Site 3
(Ledger)	(Committed)	(Committed)	(Committed)	(Committed)

4.2. Quy trình xử lý lỗi Byzantine Equivocation (Byzantine Equivocation Flow)

Mục này mô tả luồng thông điệp khi hệ thống đối mặt với hành vi phá hoại độc hại (Equivocation) từ Leader (Site 0), trong khi các Replica (Site 1, 2, 3) đều trung thực và hoạt động bình thường.

4.2.1. Sơ đồ tuần tự thông điệp (Sequence Diagram)

Sơ đồ tuần tự mô tả hành vi Leader (Site 0) gửi các thông điệp bất nhất (COMMIT và ABORT) cho các Site chân/lẻ, và cách các nút trung thực truyền thông chéo để hóa giải:



4.2.2. Các bước xử lý chi tiết (Step-by-Step Processing):

- Bước 1: Leader Equivocates (Leader nói dối)**
Khi Leader (Node 0) thực hiện equivocation (gửi PRE-PREPARE có digest đúng cho Node 2 nhưng digest sai cho Node 3).

- Site 0 (Leader Byzantine) nhận giao dịch và cố tình tạo ra sự bất đồng thuận.
 - Site 0 gửi thông điệp bỏ phiếu với nội dung khác nhau đến các hàng đợi nhận tin:
 - Gửi thông điệp có phiếu **vote**: **COMMIT** cho Site 2 (Nút chặn).
 - Gửi thông điệp có phiếu **vote**: **ABORT** cho Site 1 và Site 3 (Nút lẻ).
 - Tất cả các thông điệp này đều có chữ ký số hợp lệ của Site 0 (**SIG_SITE_0**).
2. Bước 2: Bỏ phiếu và Phát chéo của Replica trung thực
- Node 2 và Node 3 nhận tin. Node 2 thấy digest đúng nên broadcast **PREPARE**(digest đúng). Node 3 nhận digest sai nên từ chối phát **PREPARE** cho digest đó.
- Các Replica trung thực (Site 1, 2, 3) nhận thông điệp từ Leader. Vì giao dịch là hợp lệ về mặt dữ liệu, các Replica tự quyết định bỏ phiếu **COMMIT** của mình.
 - Mỗi Replica ghi trạng thái **READY** kèm phiếu **vote**: **COMMIT** của mình vào WAL cục bộ.
 - Các Replica phát chéo phiếu **COMMIT** của mình cho toàn bộ các nút trong mạng. Site 1, 2, 3 gửi thông tin nhất quán và trung thực.

3. Bước 3: Thu thập phiếu và Tính toán Quorum

- Bước 3: Thu thập phiếu và Tính toán Quorum
 Nhờ truyền thông chéo, Node 2 nhận được **PREPARE** từ Node 1 và Node 2 (honest) cùng với Leader Node 0 -> đạt Quorum 3 phiếu **COMMIT**. Node 3 nhận được các tin nhắn **PREPARE** đúng từ Node 1 và Node 2 -> cũng đạt Quorum 3 phiếu **COMMIT** cho giao dịch đúng và cùng đưa ra quyết định **COMMIT** nhất quán. Hành vi lừa dối của Leader bị vô hiệu hóa.
 - **Site 2 (Nút nhận COMMIT từ Leader)**: Nhận được 4/4 phiếu **COMMIT** (gồm 1 từ Site 0 Byzantine, và 3 từ các site trung thực Site 1, 2, 3).
 Do $4 \geq \text{Quorum}(3) \rightarrow$ Site 2 quyết định **COMMIT**.
 - **Site 1 & Site 3 (Nút nhận ABORT từ Leader)**: Nhận được 3/4 phiếu **COMMIT** (gồm Site 1, 2, 3 trung thực) và 1 phiếu **ABORT** (từ Site 0 Byzantine). Do số phiếu **COMMIT** đạt $3 \geq \text{Quorum}(3) \rightarrow$ Site 1 và Site 3 quyết định **COMMIT**.
- Kết quả: Cả 3 nút trung thực (Site 1, 2, 3) đều đạt đủ số phiếu cam kết và cùng đưa ra quyết định **COMMIT** đồng nhất, đảm bảo Số cái hoàn toàn nhất quán.

4.2.3. Bảng phân tích phiếu bầu nhận được tại mỗi Site

Bảng dưới đây mô tả nguồn gốc các phiếu bầu mà mỗi nút trung thực thu nhận được và cách hệ thống đạt đồng thuận:

Site nhận	Phiếu từ Site 0 (Byzantine)	Phiếu từ Site 1 (Honest)	Phiếu từ Site 2 (Honest)	Phiếu từ Site 3 (Honest)	Tổng số COMMIT	Kết quả
Site 1	ABORT	COMMIT	COMMIT	COMMIT	3 / 4	COMMIT (Đạt Quorum)
Site 2	COMMIT	COMMIT	COMMIT	COMMIT	4 / 4	COMMIT (Đạt Quorum)
Site 3	ABORT	COMMIT	COMMIT	COMMIT	3 / 4	COMMIT (Đạt Quorum)

Mục này làm rõ cách thức cơ chế Quorum $2f+1$ trực tiếp hóa giải thông điệp giả mạo mà không cần sự can thiệp của con người. Bạn có thể chép trực tiếp mục này vào phần 4.2 của tài liệu Thiết kế.

5. Security Design (Thiết kế bảo mật)

Byzantine Fault Tolerance đòi hỏi một cơ chế bảo mật chặt chẽ để chống lại các lỗi độc hại có chủ đích.

Tầng mật mã học Ed25519 thực tế:

Hệ thống sử dụng mật mã học đường cong elliptic Ed25519 cho chữ ký số thực sự:

- Khóa bí mật (Private Key) được lưu giữ bởi mỗi node và client để ký số thông điệp qua hàm Ed25519.
- Khóa công khai (Public Key) được phân phối công khai để tất cả các bên có thể xác minh tính toàn vẹn của tin nhắn.
- Cơ chế bảo mật thực sự ngăn chặn hoàn toàn tấn công sửa đổi tin nhắn (Tampering) và giả mạo nguồn gửi (Spoofing) trên mạng TCP.

-
-
-

5.2. Vai trò bảo mật:

1. Chống mạo danh (Anti-Spoofing): Site Byzantine không thể giả mạo danh nghĩa của các nút trung thực để phát đi thông điệp độc hại. Ví dụ: Site 0 không thể gửi tin nhắn ghi sender: 1 kèm chữ ký giả để lừa các Site khác, vì hệ thống kiểm tra chữ ký số sẽ phát hiện chữ ký thực tế là của Site 0 hoặc không hợp lệ.
2. Đảm bảo tính toàn vẹn (Integrity): Ngăn chặn việc sửa đổi thông điệp bỏ phiếu trên đường truyền mạng giả lập.